CONFERENCE TALK · MAX HELZBERG

THE DATA LAYER CEILING

Why Your LLM's Biggest Bottleneck Isn't Your LLM

MAY 2026

About This Talk

This session draws on firsthand experience designing operational AI systems for some of the most data-complex environments in the world — financial infrastructure, manufacturing, and healthcare.

- **The claim:** In enterprise AI, the real performance ceiling is set at the data modeling layer — not the model.
- **The evidence:** Failure modes, real-world examples, and the patterns that separate systems that work from those that don't.
- **The takeaway:** Practical guidance on how data architecture decisions shape LLM and agent behavior — before a single training run begins.

CONFIDENTIAL & PROPRIETARY

PART I

THE LLM-CENTRIC FALLACY

The Dominant Narrative

The AI industry has converged on a seductive story: **better models → better outcomes.** Scale the parameters, refine the RLHF, improve the benchmark — and enterprise AI will follow.

This narrative is not wrong. It is simply incomplete — and in operational settings, dangerously so.

> **"A brilliant model reasoning over garbage data produces confident garbage."**

CONFIDENTIAL & PROPRIETARY

Performance is bounded by the weakest link.

LAYER 1

Data Modeling

Ontologies, schemas, entity resolution

THE CEILING

LAYER 2

Retrieval & Context

RAG, indexing, prompt design

LAYER 3

The Model

Parameters, fine-tuning, inference

Most enterprise AI investment flows to Layer 3. Most enterprise AI failures originate in Layer 1.

PART II

WHAT IS AN ONTOLOGY?

Ontology vs. Data Warehouse

Both store data. The similarity ends there.

DATA WAREHOUSE

- Optimized for analytical queries
- Tables and columns — rows as records
- Relationships are implicit or denormalized
- Built for human analysts and BI tools
- Context lives outside the data

DOMAIN ONTOLOGY

- Optimized for reasoning and inference
- Entities, properties, and typed relationships
- Semantic links are first-class citizens
- Built for models and agents to consume
- Meaning is encoded in the structure

CONFIDENTIAL & PROPRIETARY

Entity Resolution at Scale

Operational environments accumulate entities over years across dozens of systems. The same physical asset may appear under fourteen different identifiers.

WITHOUT RESOLUTION

```
PUMP-7A
pump_seven_alpha
P7A
Centrifugal Pump #7
Asset_00471
```

The model treats these as five different objects. Reasoning fragments. Queries fail.

WITH ONTOLOGY

```
entity: CentrifugalPump
id: pump-7a-canonical
aliases: [PUMP-7A, P7A, ...]
links: [circuit_B, maint_log]
```

One entity. Full context. Consistent reasoning across every query.

Semantic Relationships Matter

In operational domains, **how things relate is as important as what they are.** A pump failure is only meaningful when the model understands which circuit it feeds, which assets depend on it, and what the downstream impact chain looks like.



Structural

Part-of, contains, feeds-into, upstream-of



Temporal

Precedes, caused-by, scheduled-after, version-of



Operational

Responsible-for, authorized-to, escalates-to, owns

Without typed relationships, an LLM cannot reason about operational impact — it can only pattern-match on surface text.

CONFIDENTIAL & PROPRIETARY

The Ontology Is Your Best Defense Against Hallucinations

Traditional RAG retrieves text. The problem: text carries no schema, no typed relationships, no authoritative constraints — so the model can still drift. Ontology-Augmented Generation (OAG) changes the retrieval unit entirely.

TRADITIONAL RAG

User prompt

↓ retrieve text chunks

Unstructured document snippets

↓ inject into context

LLM generates response

Still guesses at meaning. No schema. No constraints.

ONTOLOGY-AUGMENTED GENERATION (OAG)

User prompt + ontology metadata

↓ LLM generates a valid object query

Structured objects + typed relationships

↓ inject objects into context

LLM reasons within the schema

When the grounding is clear, the model doesn't need to guess.

Structured Action Hierarchies

Reliable agent workflows require more than tool access. They require a semantic map of *which actions are valid in which states* — and why.

FLAT TOOL LIST	Agent selects from 40 available tools with no context on preconditions, sequencing, or authorization. Brittle, unpredictable, difficult to audit.

ACTION ONTOLOGY	Actions carry types, preconditions, permissions, and semantic tags. The agent understands *why* certain sequences make sense — and which are prohibited. Every step is traceable.

The difference between an agent that occasionally works and one that can be deployed in production.

PART III

FAILURE MODES

What goes wrong — and why.

The Three Failure Patterns

01 — Hallucination from Ambiguity

When entity boundaries are fuzzy, models fill gaps with plausible-sounding fabrications. The model isn't broken — it's doing exactly what it's designed to do with insufficient structure. **The fix is ontology grounding: inject typed**

objects and their properties directly into context, and the model reasons within a schema instead of guessing across a void.

02 — Brittle Reasoning Chains

Without semantic relationships, multi-hop reasoning collapses. The model cannot reliably answer "what is affected downstream?" if downstream relationships don't exist as explicit structure.

03 — Production Fragility

Systems that perform well in controlled tests fail silently when operational data changes. No semantic anchor means no principled failure detection.

CONFIDENTIAL & PROPRIETARY

PART IV

REAL-WORLD EXAMPLES

SECTOR

Financial Infrastructure

A financial institution deploys agents to monitor counterparty exposure and flag compliance risks across a portfolio of instruments and entities.

The problem:

Legal entities and subsidiaries exist across 12 source systems with no consistent relationship graph. The agent cannot determine that two exposures belong to the same counterparty — a critical compliance failure.

The fix:

A legal entity ontology with ownership hierarchies, jurisdictional attributes, and relationship types enables the agent to reason across subsidiary structures — and produce auditable exposure calculations.

SECTOR

Solar Panel Manufacturing

A solar manufacturer had a spare parts catalog full of ambiguous item descriptions — driving wrong orders, production delays, and wasted floor time. The solution was an LLM-powered enrichment workflow grounded entirely in the ontology.

BEFORE

Servo motor

Ambiguous. 40–50 servo motors in the catalog — impossible to distinguish at a glance.

AFTER

Motor, servo, 480 VAC, 8,000 RPM, 63mm frame

Drawn from: manufacturer, part number, OEM specs, and web search.

HUMAN-IN-THE-LOOP WORKFLOW

New value enters system → LLM generates enriched description → user reviews & approves → approved value goes live immediately. User feedback stored as a linked object, becoming a growing context corpus for future runs.

84%

ACCURACY RATE ACHIEVED

All data — items, feedback, enrichment results — lives on the same ontology object. No joins. No pipeline fragility.

CONFIDENTIAL & PROPRIETARY

SOLAR MANUFACTURING — CONTINUED

Why the Ontology Made Scaling Cheap

CONTEXT MANAGEMENT

Web search results (via Azure AI, external to Foundry) were enriched and pulled back in as linked ontology objects. Adding each new context source — manufacturer data, OEM specs, web results —

required no dataset joins. The ontology's indexing handled it. Cost of each incremental source: near zero.

PROTOTYPE → PRODUCTION

Started with a **27,000-part sample** to validate the method — including web search integration and the human feedback loop. Once validated, swapping in the full production dataset required minimal changes. The downstream ontology infrastructure stayed entirely intact.

SCHEMA CONSISTENCY

Because all part types share the same ontology object structure, the same workflow is replicable across the entire parts catalog. New part categories slot in without rebuilding the pipeline. The ontology is the reuse mechanism.

Business impact: fewer wrong orders, eliminated production delays from part mismatches, and floor users can immediately find the right part — by name.

SECTOR

Veterinary Chain Grown via M&A

Dozens of acquired practices, each with its own backend system and its own vocabulary for procedures. The same surgery had three different codes at three different hospitals. With

100,000+ procedures across the network, manual reconciliation was not an option.

ENTITY RESOLUTION PIPELINE

STEP 1	STEP 2	STEP 3	STEP 4
Embed	**Cluster**	**LLM Review**	**Human Review**
Procedure text converted to numerical vectors	Similar procedures group together in vector space	~100 procedures per batch reviewed for exact matches	Final curation validates and approves matches

EXAMPLE — TPLO SURGERY (TIBIAL PLATEAU LEVELING OSTEOTOMY)

`TPLO` + `TPLO surgery` + `Tibial Plateau Leveling Osteotomy` → **`TPLO (canonical)`**

CONFIDENTIAL & PROPRIETARY

VETERINARY CHAIN — CONTINUED

Two Ontology Layers. One Source of Truth.

LEVEL 1 — HOSPITAL LAYER

Raw procedures in natural language, exactly as each hospital's system records them. Multiple codes may map to the same procedure — that's fine at this layer. One-to-one mapping between codes and procedures is preserved for hospital-level analysis.

LEVEL 2 — NETWORK LAYER

Entity resolution runs between Level 1 and Level 2, producing a clean, standardized procedures table. Apples to apples across every hospital. This is the layer the LLM reasons over for network-wide analytics.

NETWORK PRICING IN PRACTICE

Network sets TPLO price at Level 2. The ontology automatically maps that price to each hospital's underlying Level 1 codes. Hospital systems receive their own codes — with the network decision already translated. No manual reconciliation.

WHY ONTOLOGY BEATS A CUSTOM SEMANTIC LAYER

✓ Automatic indexing — fast search across 100K+ procedures

✓ Transaction batching with rollback capabilities

✓ Built-in data lineage — every change is traceable

You could build this outside Foundry — but you'd rebuild all of this from scratch.

BUILDING THE RIGHT FOUNDATION

What a Semantic Layer Delivers

⇧ Reasoning Quality

Models operate on structured context. Relationships are explicit. Multi-hop inference is grounded — not guessed.

⇧ Traceability

Every answer can be traced to specific entities and relationships. No black box. No unexplainable outputs.

⇧ Auditability

Structured action hierarchies make agent behavior reviewable — and defensible to regulators, operators, and leadership.

⇩ Hallucinations

Reduced ambiguity means fewer gaps for the model to fill with fabrication. Confidence becomes meaningful again.

CONFIDENTIAL & PROPRIETARY

The most important AI design decisions happen before

model training begins.

01 Model your domain first. What are the canonical entities? What relationships matter? What actions are valid — and for whom?

02 Invest in entity resolution. Dirty identifiers are not a cleanup task — they are an architectural decision with downstream AI consequences.

03 Encode meaning in structure. Don't rely on the model to infer relationships that your data should make explicit.

04 Design for auditability from day one. If you can't explain how the system reached a conclusion, you can't deploy it where it matters.

The Bottom Line

You are not building an LLM application. You are building a data system that an LLM operates on top of.

The model is a reasoning engine. Give it garbage to reason over and you get sophisticated-sounding garbage back. Give it a well-structured semantic model of the domain and the model becomes dramatically more capable — not because it changed, but because the foundation finally did.

Invest in the ontology. The model will thank you.

CONFIDENTIAL & PROPRIETARY

THANK YOU

Questions & Discussion

Max Helzberg

mhelzberg@northslopetech.com

